

Exercice 1 : Ecrire un organigramme permettant de résoudre une équation du 2^e degré dans l'ensemble des nombres réels.

Program EquationDuxiemeDugre;

Var a,b,c,d:integer;

x1 ,x2 : Real;

Begin

Writeln('entrez la valeur de a b et c');

Readln(a,b,c);

If (a = 0) **then**

Writeln('L équation de premier degré')

Else

Begin

//Calculer Delta

D:= b*b-4*a*c;

If d > 0 **then**

//deux solutions réelles

begin

X1:=(-b-sqrt(D))/(2*a);

X2:=(-b+sqrt(D)) / (2*a);

Writeln ('Deux solutions réelles :',x1,'et', x2);

end

else if d=0 **then**

//une solution réelle

begin

X1:=(-b)/(2*a);

Writeln ('une solution réelle :',x1);

End

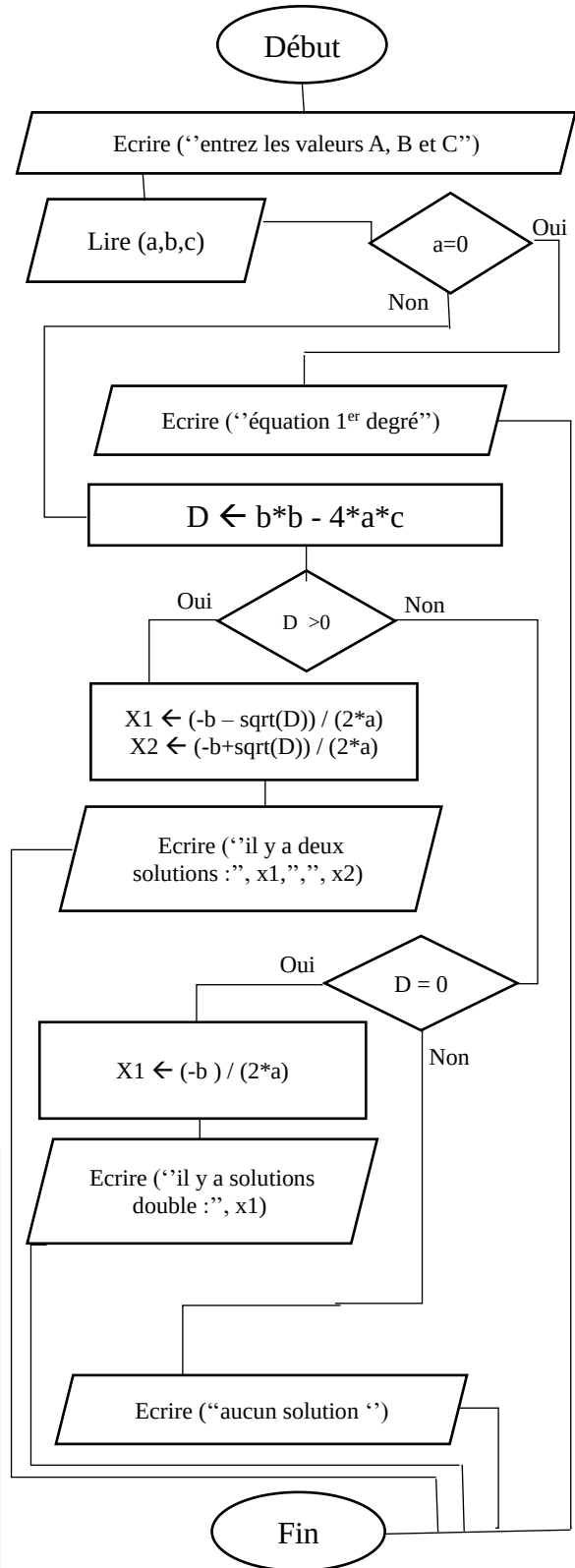
else

//pas de solution réelle

writeln ('pas de solution réelle.');

end;

End.



Exemple :

entrez la valeur de a, b et c

a	b	c	d	X1	X2
2	-1	-6			
2 < 0 alors on a faire La partie (else)					
2	-1	-6	$(-1)*(-1)-4*(2)*(-6)= 49$		
(d = 49) > 0 alors il y a Deux solution réelles					
2	-1	-6	49	$\frac{-(-1) - \sqrt{49}}{2 * 2}$ $\frac{-3}{2} = -1.5$	$\frac{-(-1) + \sqrt{49}}{2 * 2}$ $\frac{2}{2} = 2$

Deux solutions réelles : -1.5 et 2

Exercice 2 : Ecrire un algorithme qui donne la durée de vol en heure minute connaissant l'heure de départ et l'heure d'arrivée. On considère que le départ et l'arrivée ont lieu le même jour b. On suppose que la durée de vol est inférieure à 24 heures.

Program DureeVol;

Var h_depart , m_depart , h_arrive , m_arrive , h_duree , m_duree:integer;

Begin

//lire correctement l'heure de départ

Repeat

Writeln('Donner l heure départ (heure et minute)');

Readln (h_depart , m_depart);

Until (h_depart >=0) **and** (h_depart<24)**and** (m_depart>=0) **and** (m_depart < 60);

//lire correctement l'heure d'arrivée

Repeat

Writeln('Donner l heure d arrive (heure et minute)');

Readln (h_arrive , m_arrive);

Until (h_arrive >=0) **and** (h_arrive<24) **and** (m_arrive>=0) **and** (m_arrive < 60) **and** (h_depart < h_arrive);

//réglé le problème de calcul

If (m_depart > m_arrive) **then**

Begin

 m_arrive := m_arrive+60;

 h_arrive := h_arrive -1;

End;

//calculer la durée

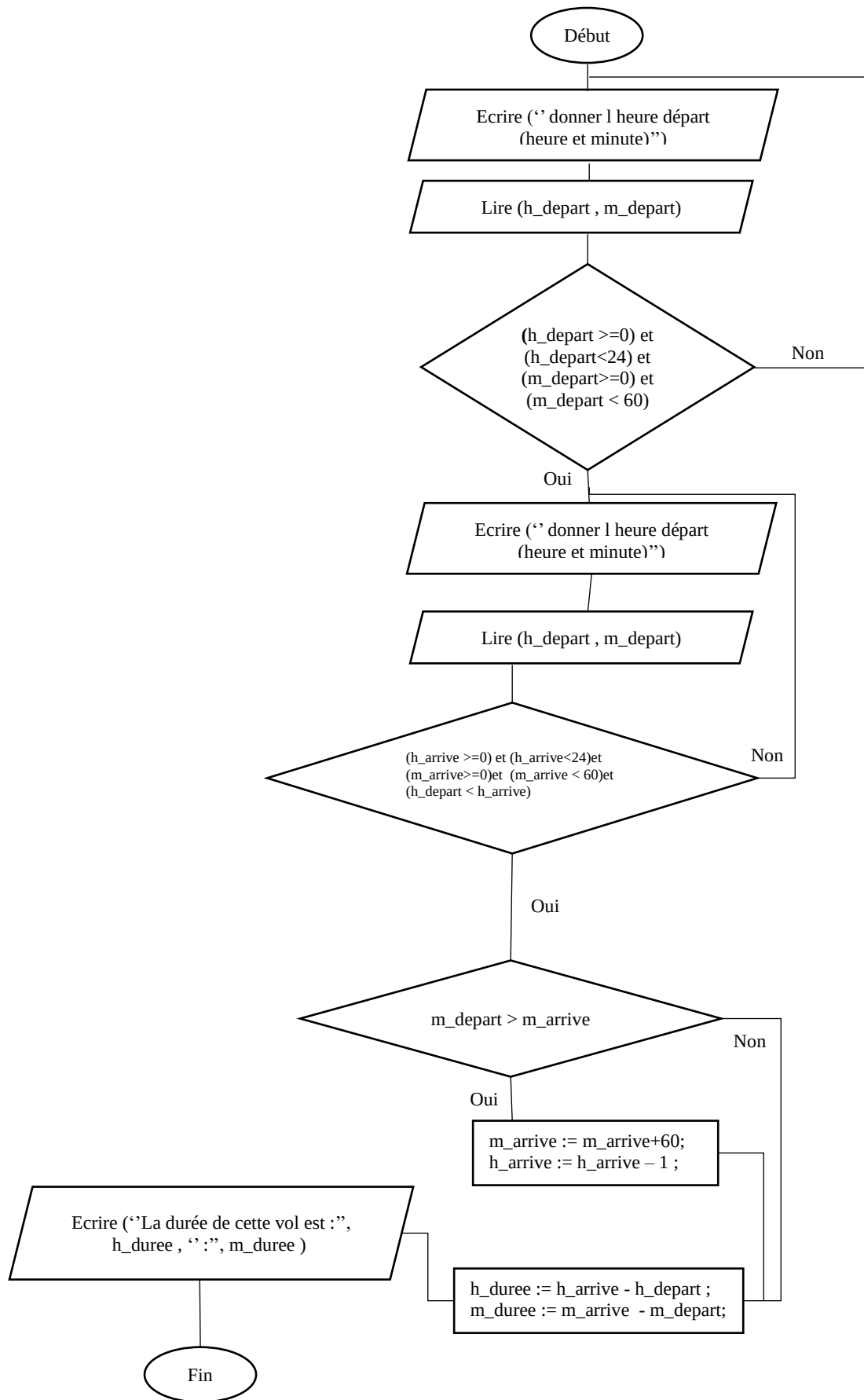
h_duree:= h_arrive - h_depart ;

m_duree:= m_arrive - m_depart;

//l'affichage

writeln ('La durée de ce vol est :', h_duree , 'h', m_duree, 'min');

End.



Exemple :

h_depart	M_depart	H_arrive	M_arrive	H_deuree	M_duree
----------	----------	----------	----------	----------	---------

Donner l heure départ (heure et minute)

h_depart	M_depart	H_arrive	M_arrive	H_deuree	M_duree
15	84				

{15>0 (true) and 15<24 (true) and 84>0 (true) and 84<60 (false)} = false
 (Tant que le résultat est faux alors on va répéter)

Donner l heure départ (heure et minute)

h_depart	M_depart	H_arrive	M_arrive	H_deuree	M_duree
15	54				

{15>0 (true) and 15<24 (true) and 54>0 (true) and 54<60 (true)} = true
 (Tant que le résultat est vrai alors on va continuer)

Donner l heure d arrive (heure et minute)

h_depart	M_depart	H_arrive	M_arrive	H_deuree	M_duree
15	54	17	10		

{17>0 (true) and 17<24 (true) and 10>0 (true) and 10<60 (true) and 15<17(true)} = true
 (Tant que le résultat est vrai alors on va continuer)

M_depart > m_arrive ? → 54 > 10 (true)

Alors :

h_depart	M_depart	H_arrive	M_arrive	H_deuree	M_duree
15	54	17-1=16	10+60=70		
				16-15=1	70-54=16

La durée de ce vol est : 1 h 16 min

Exercice 3 : Ecrire un algorithme qui lit trois valeurs entières (A, B et C) et qui permet de les trier par permutations successives et enfin les afficher dans l'ordre.

Program TrierParPermutations;

Var a,b,c,d:integer;

Begin

Writeln ('Donner trois valeurs entières');

Readln (a,b,c);

//permutation entre a et b dans le cas (a>b)

If (a > b) **then**

Begin

 d := a;

 a := b;

 b := d;

End;

//permutation entre b et c dans le cas (b>c)

If (b > c) **then**

Begin

 d := b;

 b := c;

 c := d;

End;

//permutation entre a et b dans le cas (a>b)

If (a > b) **then**

Begin

 d := a;

 a := b;

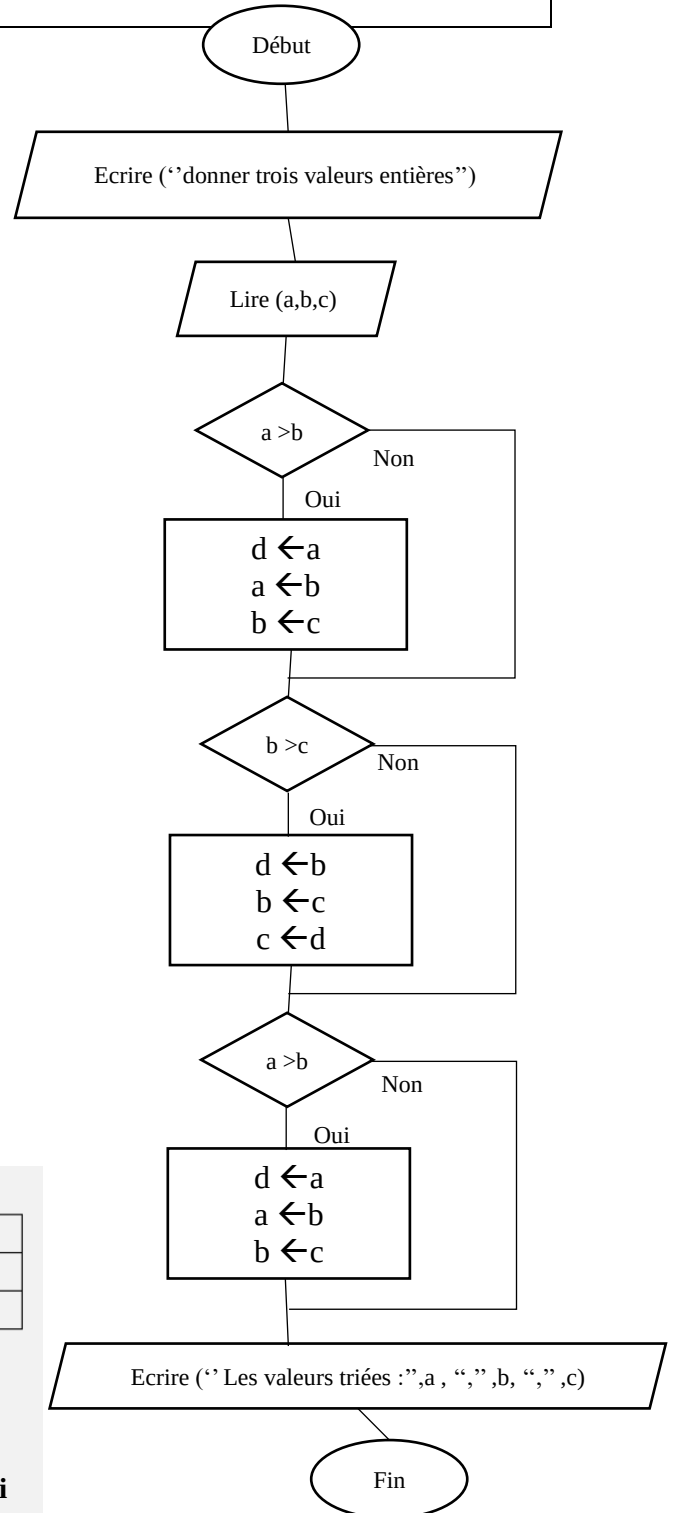
 b := d;

End;

//l'affichage

Writeln ('Les valeurs triées :',a , ',' ,b, ',' ,c);

End.



Exemple :

donner trois valeurs entières			
a	b	c	d
5	9	1	

$a > b ? \rightarrow 5 > 9$ (false)

Tant que le résultat est faux ne faire pas les instruction qui dans la condition (if)

$b > c \rightarrow 9 > 1$ (true)

Tant que le résultat est vrai alors on a faire les instrution qui dans la condition (if)

a	b	c	d
5	9 1	1 9	9

(1) (2) (3)

$a > b \rightarrow 5 > 1$ (true)

Tant que le résultat est vrai alors on à faire les instructions qui dans la condition (if)

a	b	c	d
5 1	1 5	9	9 5

(1) (2) (3)

Les valeurs triées : 1, 5, 9

Exercice 4 : Ecrire un algorithme calculatrice permettant la saisie du premier entier (a) de l'opération (+ ou – ou * ou / : sont des caractères) et du deuxième entier (b) et qui affiche le résultat.

Program Calculatrice;

Var a,b : integer;

operation: char;

begin

Writeln ('Entrez le premier entier');

Readln(a);

Writeln ('Entrez l'opérateur (+,-,*, /)');

Readln (operation);

Writeln ('Entrez le deuxième entier');

Readln(b);

Case operation **of**

'+' : **writeln** (a , operation , b, ' = ',a+b);

'-' : **writeln** (a , operation , b, ' = ',a-b);

'*' : **writeln** (a , operation , b, ' = ',a*b);

'/' : **If** (b = 0) **then**

writeln ('La division sur zéro n est pas autorisée')

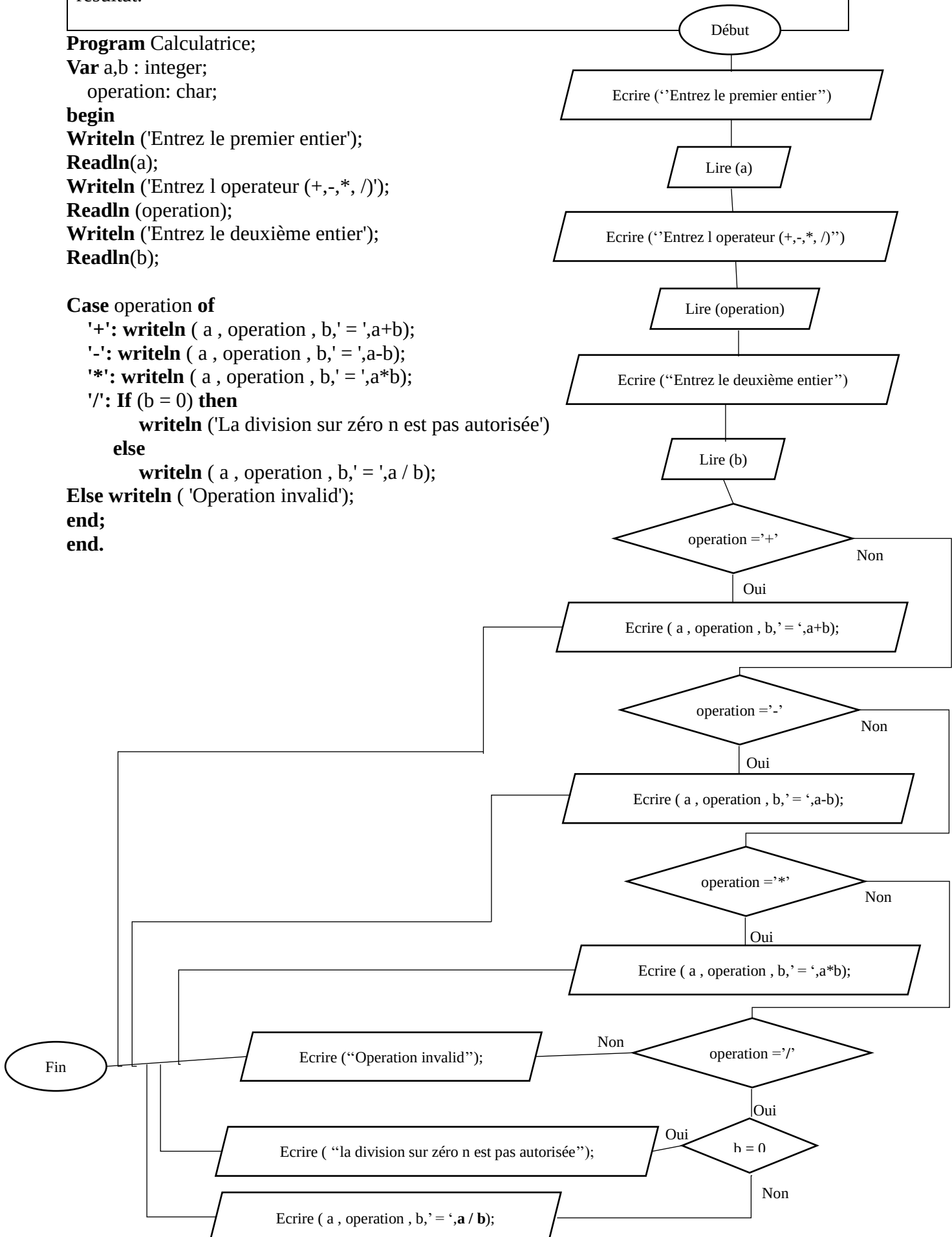
else

writeln (a , operation , b, ' = ',a / b);

Else writeln ('Operation invalid');

end;

end.



Exemple 1 :

a (integer)	operation (char)	b (integer)
Entrez le premier entier		
a	operation	b
8		
Entrez l'opérateur (+, -, *, /)		
a	operation	b
8	-	
Entrez le deuxième entier		
a	operation	b
8	-	2
8 - 2 = 6		

Exemple 2 :

a (integer)	operation (char)	b (integer)
Entrez le premier entier		
a	operation	b
8		
Entrez l'opérateur (+, -, *, /)		
a	operation	b
8	n	
Entrez le deuxième entier		
a	operation	b
8	n	1
Operation invalid		

Exemple 3 :

a (integer)	operation (char)	b (integer)
Entrez le premier entier		
a	operation	b
10		
Entrez l'opérateur (+, -, *, /)		
a	operation	b
10	/	
Entrez le deuxième entier		
a	operation	b
10	/	0
La division sur zéro n'est pas autorisée		

Exercice 6 : Ecrire un organigramme qui calcul le PGCD de deux nombres a et b.

Indication : le PGCD (28, 16) = PGCD (16, 28-16=12) = PGCD (12, 4) = PGCD(8,4) = PGCD(4,4) = 4 (lorsque les deux nombres seront égaux, on aura obtenu le PGCD dont la plus basse valeur possible sera 1.

Solution 1 : (division)

Program CalculPGCD;

Var a,b,i,c,d:integer;

Begin

Writeln ('Entrez les deux nombres');

Readln (a,b);

//gardé les valeurs initial de a et b dans c et d pour utiliser plus tard dans l'affichage

 c := a;

 d := b;

//On à sortir a la boucle jusqu'à b=0

While b <>0 **do**

Begin

 i:=a mod b;

 a:= b;

 b:= i;

End;

//L'affichage

Writeln ('PGCD('c ','d,')='a);

End.

Repeat

 i := a mod b;

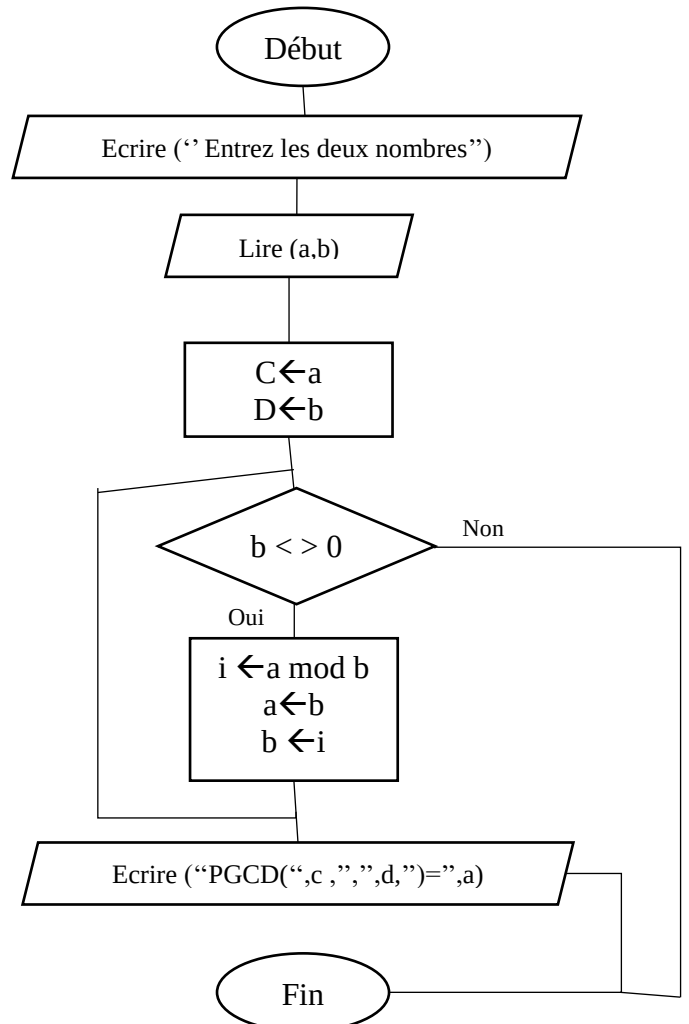
 a:= b;

 b:= i;

until b=0;

Exemple :

Entrez les deux nombres				
i	a	b	C	D
	81	15		
	81	15	81	15
81 mod 15=6	15	6	81	15
15 mod 6 = 3	6	3	81	15
6 mod 3 = 0	3	0	81	15
Tant que b=0 on a Sortir a la boucle « while »				
PGCD(81,15)=3				



Solution 2 : (soustraction)**Program** CalculPGCD;**Var** a,b,i,c,d:integer;**Begin** **Writeln** ('Entrez les deux nombres'); **Readln** (a,b);**//gardé les valeurs initial de a et b dans c et d pour utiliser plus tard dans l'affichage**

c := a;

d := b;

//On à sortir a la boucle jusqu'à b=0 **While** a <> b **do** **If** a > b **then**

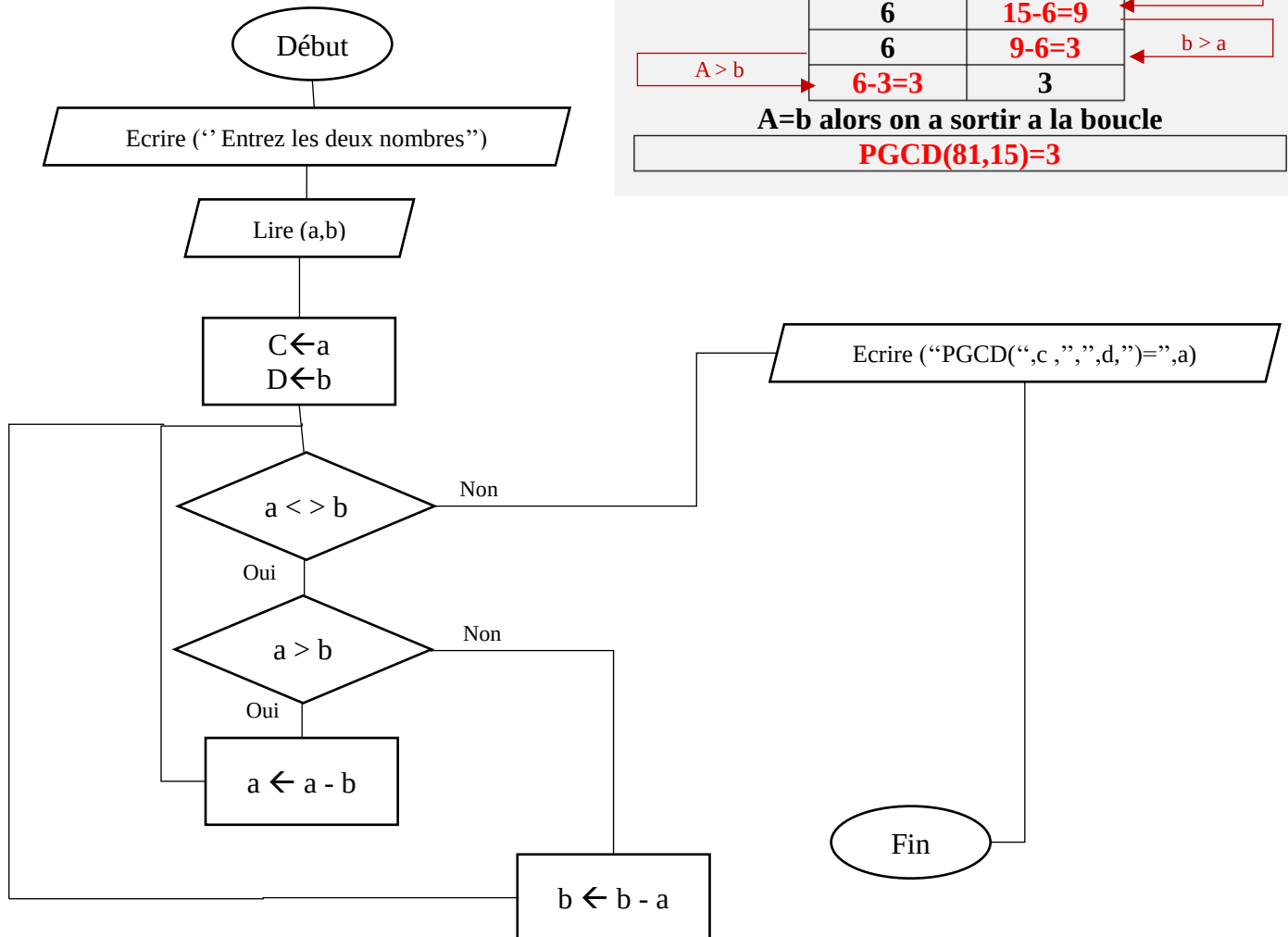
a := a-b

else

b := b-a;

//L'affichage **Writeln** ('PGCD(' ,c ,',' ,d,')=' ,a);**End.****Exemple : a=81 et b=15**

Entrez les deux nombres		
a	b	
81	15	
81-15=66	15	A > b
66-15=51	15	A > b
51-15=36	15	A > b
36-15=21	15	A > b
21-15=6	15	A > b
6	15-6=9	b > a
6	9-6=3	b > a
6-3=3	3	A > b
A=b alors on a sortir a la boucle		
PGCD(81,15)=3		



Exercice 7 : Ecrire un organigramme qui vérifie si un nombre est un carré parfait ou non.
55 ? non ; 81 ? Oui (9x9)

Solution1 : (variable booléen)

Program CarreeParfait;

Var nombre, racine: integer;
Parfait: Boolean;

Begin

Writeln ('Entrez un nombre');

Readln (nombre);

parfait := false;

//on repart de zéro en résolvant le cas si l'utilisateur
saisit la valeur (0 c'est un nombre positive)

racine :=0;

While (parfait = false) **and** (racine*racine <=nombre) **do**

Begin

If (racine*racine=nombre) **then**

parfait:=true

else

racine := racine+1;

End;

If parfait=true **then**

Writeln('Oui,',nombre, ' c est un Carré parfait(',racine,',*',racine,')')

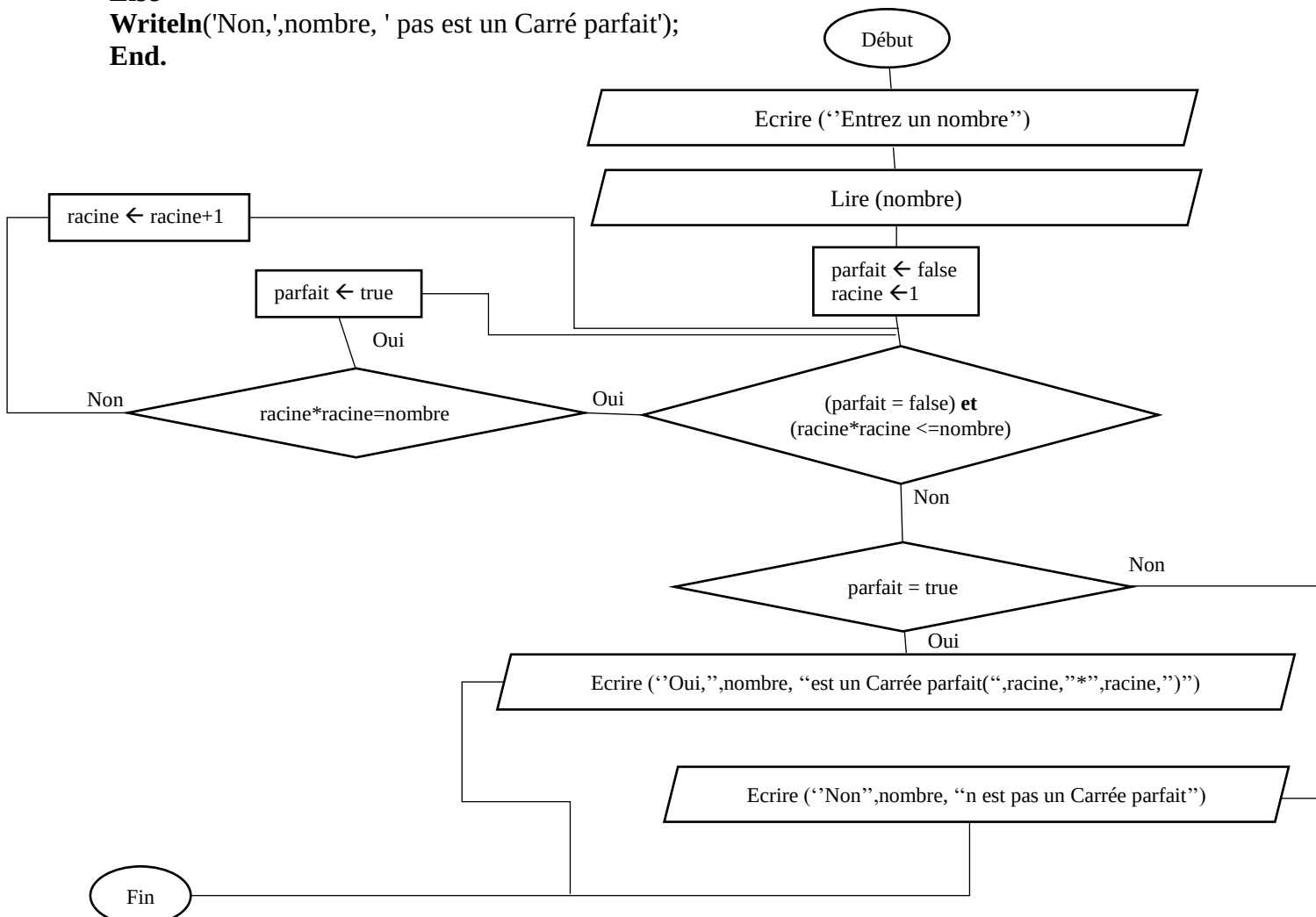
Else

Writeln('Non,',nombre, ' pas est un Carré parfait');

End.

Exemple :

Entrez un nombre		
nombre	racine	parfait
4		
4		false
4	1	false
(parfait = false) and (1*1 <=4) → vrai Entrer dans la boucle « while »		
(1*1=4)! → faux Entrer dans "else"		
4	1+1=2	false
(parfait = false) and (2*2<=4) → vrai Entrer dans la boucle « while »		
(2*2=4)! →vrai Entrer dans "if"		
4	2	true
(parfait = true) and (3*3<= 9) → faux N'Entre pas dans la boucle « while »		
Oui, 4 est un Carrée parfait(2*2)		



Solution2 : (sans utilise un variable booléen)

Program CarreeParfait;

Var nombre, racine: integer;

Begin

Writeln ('Entrez un nombre');

Readln (nombre);

//on repart de zéro en résolvant le cas si

l'utilisateur saisit la valeur

(0 c'est un nombre positive)

racine :=0;

While (racine*racine < nombre) **do**

racine := racine+1;

If racine*racine=nombre **then**

Writeln('Oui,',nombre, ' c est un Carré parfait(',racine,',',racine,')')

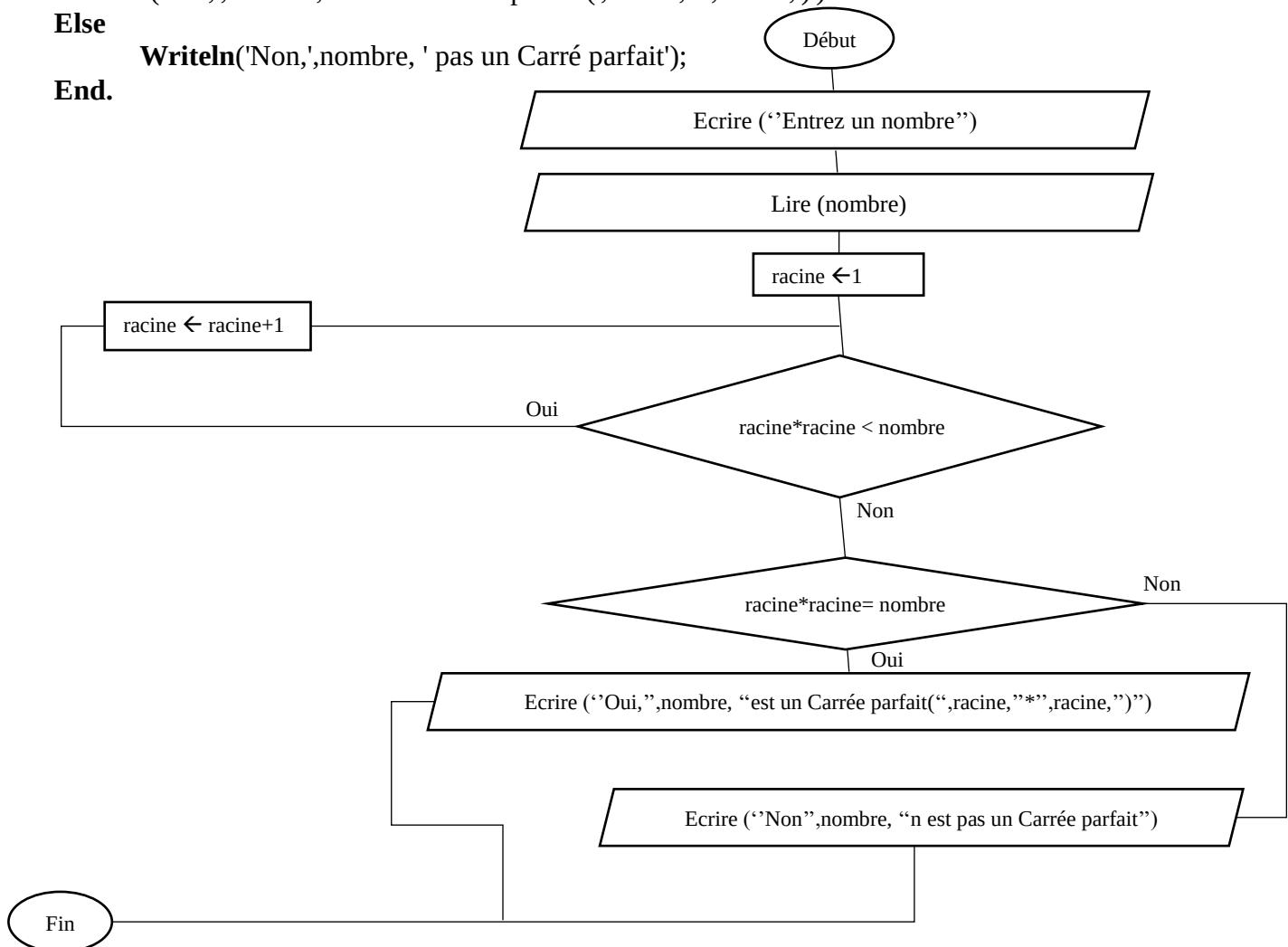
Else

Writeln('Non,',nombre, ' pas un Carré parfait');

End.

Exemple :

Entrez un nombre	
nombre	racine
4	
4	1
(1*1 < 4) → vrai Entrer dans la boucle « while »	
4	1+1=2
(2*2 < 4) → faux N'Entre pas dans la boucle « while »	
(2*2=4) → vrai Entrer dans "if"	
Oui, 4 est un Carrée parfait(2*2)	



Exercice 8 : Ecrire un programme qui vérifie si un nombre est premier.

Solution 1 :(variable booléen)(la boucle while)

Program NombrePremier;

Var nombre, cpt: integer;

Premier: Boolean;

Begin

Writeln('Entrez un nombre');

Readln(nombre);

Premier := true;

Cpt := 2;

If nombre <= 1 **then** premier := false

Else

Begin

While (cpt*cpt <= nombre) **and** premier=true **do**

Begin

If (nombre mod cpt)=0 **then**

Premier := false

Else

Cpt:=cpt+1;

End;

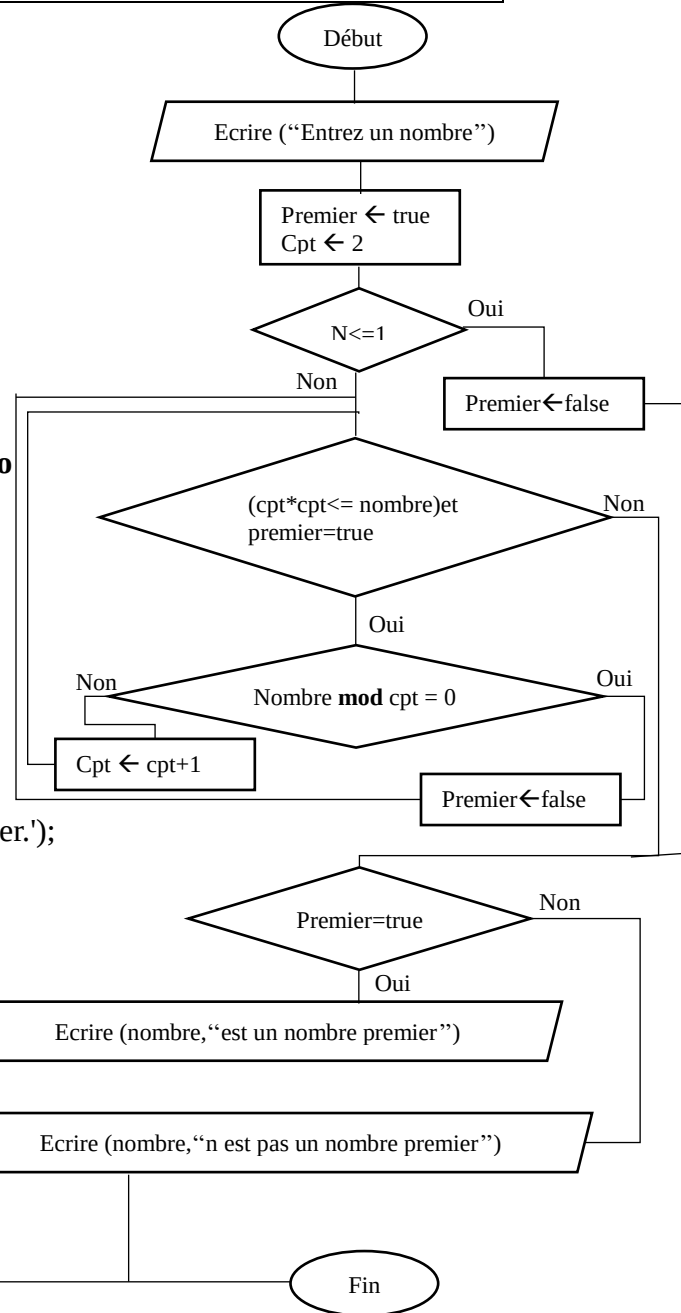
End;

If premier=true **then**

Writeln (nombre,' c est un nombre premier.')

Else writeln (nombre,' ce n est pas un nombre premier.');

End.



Exemple :

Entrez un nombre

nombre	Premier	cpt
6		
6	true	
6	true	2
(6 <= 1) ? → false		
Alors exécutez les instructions qui dans « else »		
(2*2 <= 6 and premier=true) ? → true		
Alors exécutez les instructions qui dans « while »		
(6 mod 2 = 0) ? → true		
Alors exécutez les instructions qui dans « if »		
nombre	Premier	cpt
6	false	2
(2*2 <= 6 and premier=true) ? → false		
Alors exécutez les instructions qui après « while »		
(premier=true) ? → false		
Alors exécutez les instructions qui dans « else »		
6 n est pas un nombre premier.		

Solution 2 :(sans utilisé un variable booléen)(la boucle for)

```

Program NombrePremier;
Var  nombre, cpt,i: integer;
Begin
Writeln('Entrez un nombre');
Readln(nombre);
Cpt := 0;
If nombre <= 1 then
    writeln (nombre,' ce n est pas un nombre premier.')
Else
Begin
    For i := 2 to nombre div 2 do
        If (nombre mod i)=0 then Cpt:=cpt+1;
    If cpt=0 then
        Writeln (nombre,' c est un nombre premier.')
    Else
        writeln (nombre,' ce n est pas un nombre premier.');
```

End;
End.

Exemple 1 :

Entrez un nombre		
nombre	i	cpt
7		
7		0
(7<=1) ? → false		
Alors exécutez les instructions qui dans « else »		
7	2	0
(7 mod 2 =0) ? →false		
Alors n'exécute pas les instructions qui dans « if »		
7	3	0
(7 mod 3 =0) ? →false		
Alors n'exécute pas les instructions qui dans « if »		
7	4	0
(i=4) > (7 div 2 =3)		
Alors on a quitté la boucle for		
(cpt=0) ? → true		
Alors exécutez les instructions qui dans « if »		
7 est un nombre premier.		

Exemple 2:

Entrez un nombre		
nombre	i	cpt
6		
6		0
(6<=1) ? → false		
Alors exécutez les instructions qui dans « else »		
6	2	0
(6 mod 2 =0) ? →true		
Alors exécutez les instructions qui dans « if »		
6	2	0+1=1
6	3	1
(6 mod 3 =0) ? →true		
Alors exécutez les instructions qui dans « if »		
6	3	1+1=2
6	4	0
(i=4) > (6 div 2 =3)		
Alors on a quitté la boucle for		
(cpt=0) ? → false		
Alors exécutez les instructions qui dans « else »		

